

In the name of ALLAH



**Amirkabir University of Technology
(Tehran Polytechnic)**

Machine Learning Course By Dr. Nazerfard

CE5501 | Fall 2024

Assignment (3.4)

Mohammad Sadegh Eftekhari

403131014

Sadegh.eft@aut.ac.ir

Contents

Before starting with the tasks.....	2
Imports	2
Loading the data.....	3
Task 'a': with at least 3 values for the below parameters, train the random forest classifier on the dataset	6
Task 'b': analyze the effects of the parameters on the model performance.....	7
Taks 'c': use any of the ensemble methods and try to achieve a better accuracy on the test dataset	8

Before starting with the tasks

Imports

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,
GradientBoostingClassifier, BaggingClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import GridSearchCV

# visualization
import seaborn as sns
import matplotlib.pyplot as plt
```

The list of libraries used for this project contains:

1. Pandas:
Used for loading/saving the dataset, working with datasets and data manipulation.
2. numpy:
numerical operations, including support for arrays and mathematical functions.
3. RandomForestClassifier
Used to classify people with and without diabetes for the first task.
4. AdaBoostClassifier & GradientBoostingClassifier & BaggingClassifier
Used as based models for the last task.
5. accuracy_score
Used for calculating accuracy.
6. GridSearchCV
Tuning our hyperparameters by performing grid search with cross validation.
7. seaborn:
For visualizing the distribution and relation between our features.
8. LogisticRegression & DecisionTreeClassifier & SVC
Used as base models.

Loading the data

```
diabet = pd.read_csv('pima_indians_diabetes.csv')
diabet.head()
```

	time_pregnant_no	plasma_concentration	diastolic_blood_pressure	triceps_skinfold_thickness	serum_insulin	bmi	diabetes_pedigree	age	class
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

lets take a look at the data types of our features

```
diabet.dtypes
```

```
time_pregnant_no      int64
plasma_concentration   int64
diastolic_blood_pressure  int64
triceps_skinfold_thickness int64
serum_insulin          int64
bmi                    float64
diabetes_pedigree       float64
age                     int64
class                  int64
```

looks like all of our features are numerical. So lets move on and check for the missing values.

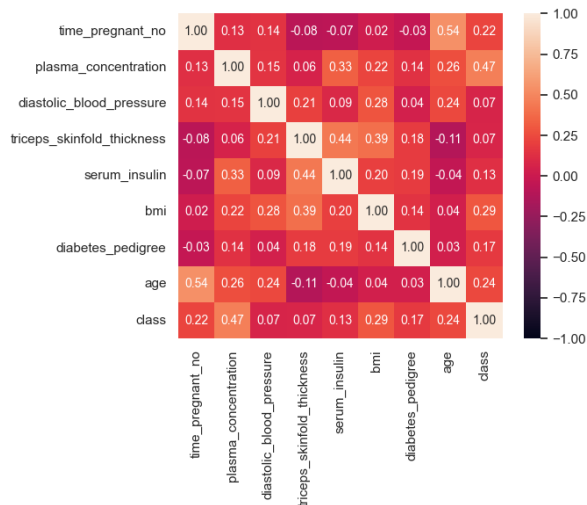
```
total = diabet.isnull().sum().sort_values(ascending=False)
percent = (diabet.isnull().sum() /
diabet.isnull().count()).sort_values(ascending=False)
missing_data = pd.concat([total, percent], axis=1, keys=["Total", 'Percent'])
missing_data
```

	Total	Percent
time_pregnant_no	0	0.0
plasma_concentration	0	0.0
diastolic_blood_pressure	0	0.0
triceps_skinfold_thickness	0	0.0
serum_insulin	0	0.0
bmi	0	0.0
diabetes_pedigree	0	0.0
age	0	0.0
class	0	0.0

no missing values as well.

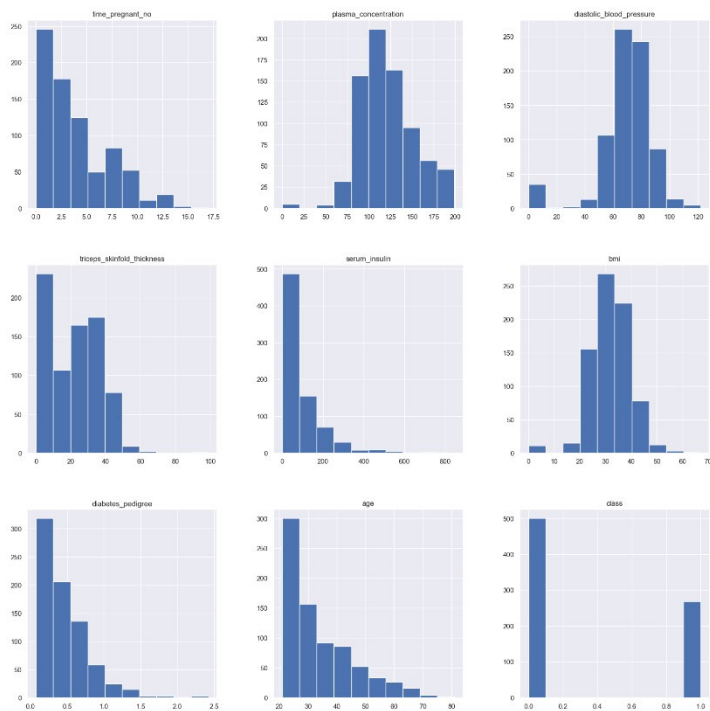
Lets do a bit of visualization, starting with heatmap

```
sns.heatmap(diabet.corr(), annot=True, annot_kws={'size': 10}, fmt='.2f',
square=True, vmax=1, vmin=-1)
```



and a histogram

```
diabet.hist(figsize = (20,20))
```



lest split the data into features and labels

```
diabet_label = diabet['class']  
diabet_feature = diabet.drop(['class'], axis=1)
```

and since the range of our variables vary so much (its obvious from the visualization), I used standard scaling here.

```
# Standard Scaling  
mean = np.mean(diabet_feature, axis=0)  
std = np.std(diabet_feature, axis=0)  
  
diabet_feature = (diabet_feature - mean) / std
```

and now splitting the data into train and test sets, for well... training and testing.

```
# split the data:  
indices = list(range(diabet.shape[0]))  
  
np.random.shuffle(indices)  
  
split_index = int(diabet.shape[0] * 0.8)  
  
train_indices = indices[:split_index]  
test_indices = indices[split_index:]  
  
x_train, x_test = diabet_feature.iloc[train_indices],  
diabet_feature.iloc[test_indices]  
y_train, y_test = diabet_label.iloc[train_indices],  
diabet_label.iloc[test_indices]
```

nothing new here, the same piece of code I used in almost every single previous project.

Task 'a': with at least 3 values for the below parameters, train the random forest classifier on the dataset

So as the problem asked for at least 3 values, I decided to make something close to 'gridsearch', picking 3 values for each parameter, and then making a nested loop using 3 loops that trains and tests the 'randomforest' 27 time, because we have 3 parameters, each with 3 values (3^3).

```
n_estimators = [100, 200, 300]
max_features = [None, 'sqrt', 'log2']
max_depth = [10, 20, None]
```

so after making 3 lists each for one parameter, we need to now write the loop.

```
best_accuracy = 0
best_params = {}

for n in n_estimators:
    for f in max_features:
        for d in max_depth:
            rf = RandomForestClassifier(n_estimators=n, max_features=f,
max_depth=d, random_state=42)
            rf.fit(x_train, y_train)

            y_train_pred = rf.predict(x_train)
            y_test_pred = rf.predict(x_test)

            train_accuracy = accuracy_score(y_train, y_train_pred)
            test_accuracy = accuracy_score(y_test, y_test_pred)

            print(f'n_estimators: {n}, max_features: {f}, max_depth: {d}')
            print(f'Training Accuracy: {train_accuracy:.2f}, Test Accuracy:
{test_accuracy:.2f}\n')

            if test_accuracy > best_accuracy:
                best_accuracy = test_accuracy
                best_params = {'n_estimators': n, 'max_features': f, 'max_depth':
d}

print(f'*****Best Parameters: {best_params}*****')
print(f'*****Best Test Accuracy: {best_accuracy:.2f}*****')
```

since the best accuracy and best parameters are going to be kept through out the whole process, we define them outside the loops, now the loops I think are pretty straight forward, it does what we see in almost every machine learning project, picks the value for each parameter, defines an object, trains, predicts and then checks the results. The only thing new here, is we keep track of the best parameters and best accuracy through out the process. And here is the output:

```
n_estimators: 100, max_features: None, max_depth: 10
Training Accuracy: 1.00, Test Accuracy: 0.80

n_estimators: 100, max_features: None, max_depth: 20
Training Accuracy: 1.00, Test Accuracy: 0.78

n_estimators: 100, max_features: None, max_depth: None
Training Accuracy: 1.00, Test Accuracy: 0.78

n_estimators: 100, max_features: sqrt, max_depth: 10
Training Accuracy: 1.00, Test Accuracy: 0.79

n_estimators: 100, max_features: sqrt, max_depth: 20
Training Accuracy: 1.00, Test Accuracy: 0.78

n_estimators: 100, max_features: sqrt, max_depth: None
Training Accuracy: 1.00, Test Accuracy: 0.79

n_estimators: 100, max_features: log2, max_depth: 10
Training Accuracy: 1.00, Test Accuracy: 0.78

n_estimators: 100, max_features: log2, max_depth: 20
Training Accuracy: 1.00, Test Accuracy: 0.79

n_estimators: 100, max_features: log2, max_depth: None
...
Training Accuracy: 1.00, Test Accuracy: 0.80

*****Best Parameters: {'n_estimators': 300, 'max_features': 'sqrt', 'max_depth': 20}*****
*****Best Test Accuracy: 0.81*****
```

so now we know that:

Best Parameters: {'n_estimators': 300, 'max_features': 'sqrt', 'max_depth': 20}

Best Test Accuracy: 0.81

Task 'b': analyze the effects of the parameters on the model performance

The 'n_estimators' parameter sets how many trees are in the forest. The findings indicate that:

Boosting the number of estimators from 100 to 300 gave a small bump in test accuracy. For instance, when using 'max_features'='sqrt' and 'max_depth'=20, the test accuracy went up from 0.79 with 100 estimators to 0.81 with 300 estimators. This implies that having more trees usually helps the model generalize better by lowering variance, even though the increase in accuracy was modest here.

The 'max_features' parameter determines how many features are taken into account for splitting at each node. We tried out three different values: None, sqrt, and log2.

When 'max_features' is set to None (meaning all features are used), the test accuracy was consistently lower, ranging from 0.78 to 0.79, compared to the other options. On the other hand, using 'max_features'='sqrt' showed better results, particularly with higher 'n_estimators' and 'max_depth' values, hitting the best accuracy of 0.81. The log2 setting performed similarly to sqrt but didn't quite reach the top accuracy. This suggests that by limiting the number of features (using either sqrt or log2), the model can generalize better, adding some diversity to the trees.

The 'max_depth' setting limits how deep the trees can grow. When set to 10, the accuracy usually dropped, hinting that the trees weren't capturing enough complexity. However, letting the trees go deeper, like setting it to 20 or leaving it unlimited, boosted the test accuracy. The sweet spot was found at 'max_depth'=20, where we hit a solid accuracy of 0.81, striking a good balance between underfitting and overfitting. On the other hand, when 'max_depth' was set to None, the test accuracy mostly stayed around 0.79, probably because of overfitting issues.

The training accuracy was always 1.00 for all parameter combinations, showing that the model was a perfect match for the training data. But the differences in test accuracy point out how crucial it is to finetune the parameters for improved generalization.

Taks 'c': use any of the ensemble methods and try to achieve a better accuracy on the test dataset

I learned that the ensemble methods we use from python libraries, have default base models, so this means if I don't explicitly tell it what base models to use, its still going to use some base models by default. So I decided to first not specify the base models, and just use grid search on each of them with their default base models, and then use one of them (here I used stacking) with my own base models.

Like the last time we used the grid search, we define a dictionary and set different values for each parameter of each model.

```
# Define parameter grids for each model
param_grids = {
    'Bagging': {
        'n_estimators': [10, 20, 50, 100],
    },
    'AdaBoost': {
        'n_estimators': [50, 100, 200],
        'learning_rate': [0.01, 0.1, 0.5],
    },
    'Gradient Boosting': {
        'n_estimators': [50, 100, 200],
        'learning_rate': [0.01, 0.1, 0.2],
        'max_depth': [3, 5, 7],
    }
}
```

We then define the models themselves.

```
Bagging = BaggingClassifier(estimator=RandomForestClassifier(random_state=42),
random_state=42)
AdaBoost = AdaBoostClassifier(random_state=42)
GBoost = GradientBoostingClassifier(random_state=42)
```

Now we also need a results dictionary to keep our best results for each model, and a dictionary to hold our defined models, so we don't need to manually train and test each of them individually. This way we can just loop over the dictionary instead.

```
results = {}
models = {
    'Bagging': Bagging,
    'AdaBoost': AdaBoost,
    'Gradient Boosting': GBoost
}
```

And here comes the, training and testing part:

```
#Grid Search for each model
for model_name, model in models.items():
    print(f"Tuning hyperparameters for {model_name}...")
    grid_search = GridSearchCV(
        estimator=model,
        param_grid=param_grids[model_name],
        scoring='accuracy',
        cv=5, # 5-fold
        verbose=1,
        n_jobs=-1
    )
    grid_search.fit(x_train, y_train)
    results[model_name] = grid_search.best_score_

# print optimal parameters
print("\nOptimal Parameters:")
for model_name, params in results.items():
    print(f"{model_name}: {results[model_name]}\n")
```

```
Tuning hyperparameters for Bagging...
Fitting 5 folds for each of 4 candidates, totalling 20 fits
Tuning hyperparameters for AdaBoost...
Fitting 5 folds for each of 9 candidates, totalling 45 fits
Tuning hyperparameters for Gradient Boosting...
Fitting 5 folds for each of 27 candidates, totalling 135 fits

Optimal Parameters:
Bagging: 0.7785019325603093

AdaBoost: 0.7719578835132614

Gradient Boosting: 0.775223244035719
```

we made a loop which takes the name and the model from our dictionary (the second one) to perform grid search on it, using the parameters we set up in our dictionary (the first one).

Then it trains so now the object is defined its parameters are set, we fit and then store the best results. This goes on for each of the models we defined. And then we print the best results of each.

As we can see, even though they're really close to what we got with random forest, all of them using their default base models, performed worse than the random forest model, now let's see if we can get a better results using our own base models. here I decided to use the stacking method, because it uses heterogenous models.

```

# Define 3 base models
base_models = [
    ('log_reg', LogisticRegression(max_iter=1000, random_state=42)),
    ('decision_tree', DecisionTreeClassifier(max_depth=5, random_state=42)),
    ('svm', SVC(kernel='linear', probability=True, random_state=42))
]

# Define the stacking ensemble with a meta-learner
stacking_model = StackingClassifier(
    estimators=base_models,
    final_estimator=LogisticRegression(random_state=42)
)

# Train the ensemble model
stacking_model.fit(x_train, y_train)

# predictions and accuracy
y_pred = stacking_model.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Stacking Ensemble Accuracy: {accuracy}")

```

```
Stacking Ensemble Accuracy: 0.7662337662337663
```

ok it didn't turn out the way I wanted to, I even scored less than the models I didn't explicitly tell what to use as base models. I believe there are things I can do to maybe improve on it, but I don't want to spend too much time on this one problem, there I have to study for my exams after all :)